Rapport : gestion des livraisons

1. Introduction

Le programme gère les colis d'une entreprise de livraison et prépare le chargement d'un camion. On l'a écrit en C : comme on code tout nous-mêmes, on peut expliquer ce que fait chaque partie et ce qu'elle coûte.

Dans la suite, n est le nombre de colis et C la capacité du camion en kg. La priorité 1 est la plus urgente.

2. Choix algorithmiques

2.1 Stocker les colis : un tableau qui s'agrandit

Élément	**Notre choix**
Problème	On ne sait pas combien de colis on aura.
Solution	Un tableau de 16 cases au départ, qu'on double avec `realloc` quand il est plein.
Temps	Ajout en O(1) en moyenne.
Espace	O(n), au pire 2 fois la place utile.
Alternative	Une liste chaînée ou un tableau de taille fixe.
Justification	On accède directement à chaque case, ce qui sert pour le tri, et le tableau s'adapte au nombre de colis.

2.2 Trier les colis : le tri fusion

Élément	**Notre choix**
Problème	Afficher les colis par poids, par valeur ou par priorité.
Solution	On coupe le tableau en deux, on trie chaque moitié, puis on les fusionne. On a un seul tri pour les 3 critères : on lui donne la fonction qui compare deux colis.
Temps	O(n log n) dans tous les cas.
Espace	O(n) pour le tableau temporaire.
Alternative	Le tri à bulles, en $O(n^2)$, est trop lent. Le tri rapide prend moins de mémoire mais peut tomber en $O(n^2)$.
Justification	Il est toujours rapide, il n'a pas de mauvais cas. On accepte d'utiliser un peu plus de mémoire pour ça.

2.3 Remplir le camion : le sac à dos

Élément	Notre choix
Problème	Choisir les colis qui donnent la plus grande valeur sans dépasser la capacité.
Solution	La programmation dynamique. La case `meilleur[i][w]` contient la meilleure valeur possible avec les i premiers colis et w kg. Pour chaque colis on garde le meilleur cas, puis on remonte le tableau pour retrouver les colis choisis.
Temps	O(n × C).
Espace	O(n × C).
Alternative	Tester toutes les combinaisons coûte O(2^n), c'est impossible. La méthode gloutonne est rapide mais ne donne pas toujours le meilleur résultat.
Justification	C'est la seule méthode qui donne toujours la meilleure valeur en un temps raisonnable.

2.4 Tenir compte de la priorité

Élément	Notre choix
Problème	Favoriser les colis urgents, sans formule imposée.
Solution	Chaque colis a un score égal à `valeur × (6 - priorité)`, et on lance le même sac à dos sur ce score. Un colis de priorité 1 compte donc 5 fois sa valeur, un colis de priorité 5 une seule fois.
Temps	O(n × C), comme avant.
Espace	O(n × C), comme avant.
Alternative	Charger d'office les colis de priorité 1, ou ajouter un bonus à la valeur (difficile à régler).
Justification	Le poids reste la limite, la valeur compte toujours et la priorité la multiplie. Un colis urgent passe devant sans être imposé.

3. Quand le nombre de colis augmente

Temps mesurés avec `mesures.c`, en millisecondes :

n	Ajout de n colis	1 000 recherches	Tri	Camion 50 kg	Camion 5 000 kg
100	0	0	0	0	3
1 000	0	2	0	1	39
10 000	96	20	2	3	377
100 000	10 533	199	25	33	3 344

Ajout des colis, $O(n^2)$. À chaque ajout on parcourt tout le tableau pour vérifier que l'identifiant n'existe pas. Quand n est multiplié par 10, le temps est multiplié par 100 : il faut 10 secondes pour 100 000 colis, ce n'est plus adapté. On pourrait utiliser une table de hachage qui donne directement la position d'un colis à partir de son identifiant, pour vérifier en O(1).

Recherche d'un colis, O(n). Le temps suit n, soit 0,2 ms par recherche pour 100 000 colis. Ça reste correct pour l'utilisateur, et la même table de hachage l'améliorerait aussi.

Remplissage du camion, O(n × C). Le temps et la mémoire augmentent avec n mais aussi avec C. Pour 100 000 colis et 5 000 kg, le tableau prend 1,9 Go, ce qui devient limite. On pourrait ne garder qu'une ligne du tableau, ou passer à la méthode gloutonne pour les très gros camions.

Le tri reste rapide, il tient bien la charge.